Módulo 4: Versionamiento

> **Enunciado.** Estoy trabajando en `feature/new-login`. Un compañero fusionó en `main` un cambio crítico que corrige un bug de seguridad, y necesito incorporar **solo ese cambio** a mi rama, sin el resto de `main`. ¿Qué comando o estrategia usaría y por qué?

Respuesta corta. Uso `git cherry-pick -x <hash-del-fix>`. Aplica en mi rama únicamente los cambios de ese commit, como un commit nuevo que referencia al original. `git merge main` y `git rebase main` traerían **todo** lo nuevo de `main`, que es justo lo que se quiere evitar.

1. Por qué cherry-pick

Opción	¿Trae solo el fix?	Comentario
`git cherry-pick -x <hash>`	**Sí**	Copia el *diff* de ese commit; `-x` deja trazabilidad al commit original
`git merge origin/main`	No	Integra todos los commits nuevos de `main`
`git rebase origin/main`	No	Mueve la rama sobre todo `main` y además reescribe la historia (requiere *force push*)

2. Paso a paso

```
# 1. Actualizar las referencias remotas (sin tocar mi rama)
git fetch origin

# 2. Identificar el commit del fix en main (por mensaje, autor, PR o archivo)
git log origin/main --oneline --grep="security"
git log origin/main --oneline -- src/auth/          # o por la ruta que corrige
git show <hash>                                     # revisar qué cambia exactamente

# 3. Aplicarlo en mi rama, con trazabilidad
git switch feature/new-login
git cherry-pick -x <hash>
#   el mensaje queda: "... (cherry picked from commit <hash>)"

# 4. Verificar: compilar y correr las pruebas (incluidas las de seguridad)
./mvnw verify            # o npm test, según el proyecto

# 5. Publicar: es un commit nuevo encima de mi rama, no requiere --force
git push
```

3. Casos especiales

Caso	Qué hacer
El fix llegó como *merge commit* (PR con *merge*)	`git cherry-pick -x -m 1 <hash-del-merge>`. `-m 1` indica que el diff se calcula contra el primer padre (la línea de `main`), es decir, se aplican todos los cambios que trajo ese PR.
Fueron varios commits	Rango consecutivo: `git cherry-pick -x A^..B` (incluye A y B; `A..B` excluiría A). Si no son consecutivos: `git cherry-pick -x <h1> <h2> <h3>`, en orden cronológico.
Hay conflictos	Git se detiene. Resuelvo los archivos, `git add <archivos>` y `git cherry-pick --continue`. Para deshacer todo, `git cherry-pick --abort`; para saltar un commit de un rango, `git cherry-pick --skip`.
El PR se integró con *squash merge*	Es el caso más simple: el fix es **un solo commit normal** en `main`. Se aplica con `git cherry-pick -x <hash>` sin `-m`.

4. Riesgos y buenas prácticas

- **Dependencias ocultas.** El fix puede usar código introducido por *otros* commits de `main` (una función utilitaria o una nueva versión de una librería). Por eso reviso `git show <hash>`, compilo y corro las pruebas antes de publicar. Si depende de otros cambios, los traigo también, de forma explícita y mínima.
- **El commit queda "duplicado" (con otro hash), y Git lo maneja.** Cuando la feature se integre en `main`, el merge de tres vías ve el mismo cambio en ambos lados y no genera conflicto. Si en lugar de merge se hace `git rebase main`, Git detecta que el parche ya existe (mismo *patch-id*) y lo omite. `git cherry -v main feature/new-login` marca con `-` los commits equivalentes. La demo (§6) lo comprueba. Solo puede haber conflicto si en `main` volvieron a modificar esas mismas líneas después del fix.
- **Trazabilidad con `-x`.** Cualquiera que vea el commit en la feature sabe de dónde viene; sirve en auditorías de seguridad.
- **Comunicación.** Lo menciono en la descripción del PR de la feature ("incluye cherry-pick de `<hash>`, fix SEC-123") para que el revisor no lo confunda con trabajo propio de la rama.
- **Pruebas de seguridad.** Además de las pruebas unitarias, corro las pruebas o el escaneo que cubren la vulnerabilidad (SAST o la prueba de regresión que haya agregado el fix).
- **No reemplaza la integración normal.** Más adelante la feature se actualizará con `main` o se integrará por PR; el cherry-pick resuelve la urgencia, no sustituye ese flujo.

5. Alternativas y por qué no

Alternativa	Por qué no en este caso
`git merge origin/main`	Trae todo `main`: cambios ajenos y quizá inestables para la feature. Es válida cuando sí se quiere actualizar la rama completa, pero no es lo que pide el escenario.
`git rebase origin/main`	Igual que el merge en alcance, y además reescribe la historia de una rama ya publicada (*force push*, que afecta a quien comparta la rama).
`git checkout origin/main -- archivo`	Copia el **archivo completo** como está en `main`, incluidas otras modificaciones a ese archivo que no son del fix. Pierde autor, mensaje y trazabilidad.
`git diff A B | git apply` o `format-patch` / `am`	Funciona, pero es una versión manual de lo que `cherry-pick` ya hace con metadatos, detección de conflictos y referencia al original. Útil solo entre repositorios distintos.

6. Demo reproducible

Scripts: `demo-cherry-pick.sh` (Linux, macOS, Git Bash) y `demo-cherry-pick.ps1` (Windows PowerShell). Cada uno:

1. Crea un repositorio temporal.
2. Simula el escenario: `main` recibe un cambio no relacionado, el fix de seguridad y otro cambio no relacionado, mientras la feature avanza.
3. Aplica el cherry-pick y **verifica con aserciones** el resultado.
4. Borra el repositorio al terminar.

Ejecuté ambos el 2026-10-15 con Git 2.55.0 en Windows 11. Los dos pasaron las 6 verificaciones; las salidas completas están en `salida-demo-bash.txt` y `salida-demo-powershell.txt`. Salida real de `./demo-cherry-pick.sh`:

```
=== 3. Mientras tanto, main recibe tres commits (solo uno es el fix de seguridad) ===
619bbf5 chore: actualización de dependencias (NO debe llegar a la feature)
53da4f6 fix(security): rechazar tokens expirados (CVE interno SEC-123)
7581f66 feat(reportes): módulo de reportes (NO debe llegar a la feature)
87bdf6b feat: autenticación inicial

=== 4. Identificar el commit del fix en main (por mensaje) ===
53da4f6 fix(security): rechazar tokens expirados (CVE interno SEC-123)

 auth.js | 4 +++-
 1 file changed, 3 insertions(+), 1 deletion(-)

=== 5. Aplicar SOLO el fix en la feature: git cherry-pick -x ===
[feature/new-login d7d32c6] fix(security): rechazar tokens expirados (CVE interno SEC-123)
 Date: Thu Oct 15 22:20:11 2026 -0500
 1 file changed, 3 insertions(+), 1 deletion(-)
* d7d32c6 fix(security): rechazar tokens expirados (CVE interno SEC-123)
* ed8c64c feat(login): opción recordar sesión
* 18a17c7 feat(login): nuevo diseño de la pantalla de login
| * 619bbf5 chore: actualización de dependencias (NO debe llegar a la feature)
| * 53da4f6 fix(security): rechazar tokens expirados (CVE interno SEC-123)
| * 7581f66 feat(reportes): módulo de reportes (NO debe llegar a la feature)
|/
* 87bdf6b feat: autenticación inicial

=== 6. Verificación ===
  [OK] la feature tiene el fix de seguridad
  [OK] la feature NO tiene reportes.js (cambio no relacionado de main)
  [OK] la feature NO tiene package.json (cambio no relacionado de main)
  [OK] el trabajo propio de la feature se conserva
  [OK] trazabilidad: el mensaje referencia el commit original (-x)

Mensaje del commit en la feature:
    fix(security): rechazar tokens expirados (CVE interno SEC-123)

    (cherry picked from commit 53da4f6b08ee3e6b34d838f1ef01f854d8e66b02)

=== 7. Más adelante: integrar la feature en main (¿se duplica el fix?) ===
git cherry main feature/new-login  ('-' = cambio equivalente ya presente en main):
    + 18a17c793bbf881041b3a2e3474bdc08f1a10a9f feat(login): nuevo diseño de la pantalla de login
    + ed8c64c3ee86219d0393d2db0eaa39c097574375 feat(login): opción recordar sesión
    - d7d32c6a8b51d6707bb0e5d03fbd693e0ff401f8 fix(security): rechazar tokens expirados (CVE interno SEC-123)
  [OK] merge sin conflictos: Git reconoce que el fix ya está aplicado
Ocurrencias de la línea del fix en auth.js de main: 1
```

(Se omitieron los pasos 1, 2 y el grafo final por brevedad; están en el archivo de salida.)

Qué demuestra: la feature recibe el commit del fix (`d7d32c6` , copia de `53da4f6`) y **no** los otros dos commits de `main` . Al integrar después la feature, `git cherry` reconoce el fix como cambio ya presente (`-`), el merge no tiene conflictos y la línea corregida aparece una sola vez.